```
y = "another value"
[1, 2, 3].each do |x|
  y = x + 1
end
[x, y]    # => ["initial value", 4]
```

Note that the assignment to the outer variable doesn't have to be executed; the Ruby interpreter just needs to have seen that the variable exists on the left side of an assignment:

```
a = "never used" if false
[99].each do |i|
  a = i
end
a         # => 99
```

The `a = i` statement in the block sets the outer value of `a` even though `a = "never used"` is not executed.

You can list *block-local variables* in the block's parameter list, preceded by a semicolon. This code does not use block-locals:

```
square = "yes"
total = 0

[1, 2, 3].each do |val|
  square = val * val
  total += square
end

puts "Total = #{total}, square = #{square}"
```

produces:

```
Total = 14, square = 9
```

In contrast, the following code, which uses a block-local variable, `square` in the outer scope is not affected by a variable of the same name within the block:

```
square = "yes"
total = 0

[1, 2, 3].each do |val; square|
  square = val * val
  total += square
end

puts "Total = #{total}, square = #{square}"
```

produces:

```
Total = 14, square = yes
```

If you are concerned about the scoping of variables with blocks, turn on Ruby warnings, and declare your block-local variables explicitly.

Pattern Matching

When you are dealing with a complicated data structure, but only need to use part of the structure, it can be awkward to access the structure through regular [] methods, with some-